

Highlights

Complexity-Constrained Few-Shot Digital Twin Model Instantiation for Heterogeneous Multi-Center Computing Networks

- Joint architecture selection and parameter adaptation enable target-specific digital twin model instantiation.
- Candidates exceeding operation-count or parameter-count limits are removed before adaptation.
- An alternative is selected only after sufficient improvement over a fixed reference.
- The analysis establishes feasibility and the minimum gain required for selecting an alternative.
- RCF-DTI reduces mean squared error by 14.60% while all selected models satisfy both limits.

Complexity-Constrained Few-Shot Digital Twin Model Instantiation for Heterogeneous Multi-Center Computing Networks

ARTICLE INFO

Keywords:

digital twin
few-shot learning
model instantiation
architecture selection
model complexity
edge–cloud computing

ABSTRACT

Shared edge–cloud digital twin platforms maintain target-specific digital twin model instances for heterogeneous computing centers to support continuous monitoring and prediction. Because edge and cloud hosting locations differ in available computing and memory capacity, each digital twin model instance operates under upper bounds on the estimated operation count per inference and the parameter count, thereby defining a complexity-constrained setting. In this setting, limited target data at a new or reconfigured center makes reliable architecture selection and parameter adaptation challenging. Existing methods usually adapt a fixed architecture or compare several architectures, but they provide limited support for joint architecture selection and parameter adaptation under these bounds, and model selection may be unreliable when validation data are limited. To address this problem, we propose the Reference-Regularized Complexity-Constrained Few-Shot Digital Twin Instantiation algorithm (RCF-DTI). RCF-DTI first removes candidates that exceed either limit and then adapts the remaining source-trained candidates using a common update budget. It selects an alternative candidate only when its validation loss is sufficiently lower than that of a fixed reference candidate. Experimental results demonstrate that RCF-DTI achieves the lowest prediction error among the compared methods, reducing mean squared error by 14.60% relative to the pretraining-and-fine-tuning baseline, while all selected models satisfy both limits.

1. Introduction

Digital twin platforms maintain virtual model instances of physical systems to support monitoring, prediction, analysis, and decision making [1–3]. In a heterogeneous multi-center computing network, a shared edge–cloud platform maintains one target-specific digital twin model instance for each physical center. Differences in monitored variables, workload patterns, and data distributions may require different model architectures. The corresponding model instances may also be hosted at edge or cloud locations with different computing and memory capacities and service requirements. Each instance therefore operates under upper bounds on the estimated operation count per inference and the parameter count, forming a complexity-constrained setting. In this setting, a new or reconfigured center may have only limited target data, making reliable architecture selection and parameter adaptation challenging. Direct reuse of an existing instance may be unsuitable because it may not fit the target center or satisfy both bounds. The platform must therefore select a suitable architecture and adapt its parameters from the available target data while ensuring that the resulting model satisfies both bounds. We call this joint process target-specific digital twin model instantiation. Figure 1 illustrates this setting.

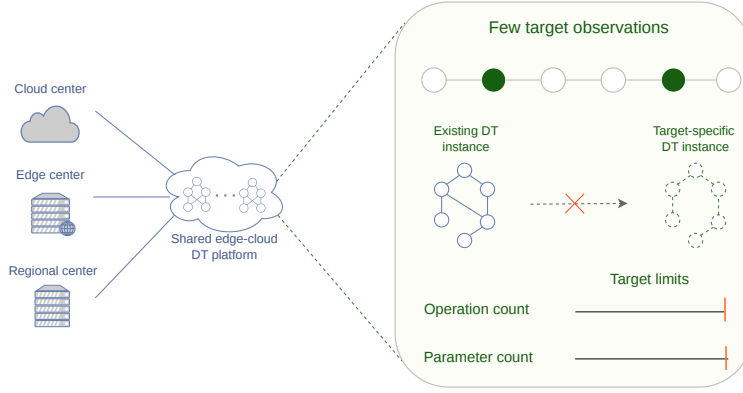


Figure 1: Platform-side target-specific digital twin model instantiation in a heterogeneous multi-center computing network. For a new or reconfigured target center, only limited target data may be available. An existing model instance may not fit the target center or satisfy the assigned upper bounds on estimated operation count and parameter count. The shared edge–cloud platform must therefore instantiate a target-specific digital twin model.

Existing studies address the main parts of this problem separately. Digital twin research supports platform-side model management, but it often assumes a predefined or common model architecture [1, 5, 7]. Few-shot and meta-learning methods transfer knowledge from related tasks, but most of them adapt one fixed architecture [9–11]. Deployment-aware architecture search considers prediction accuracy and execution cost [12, 13]. Few-shot and zero-shot search reduce repeated training, but they do not focus on reliable target-specific selection from limited validation data [14–16]. Taken together, these studies provide limited support for joint architecture selection and parameter adaptation under target-specific complexity bounds. Moreover, small validation differences may lead to unreliable model selection when validation data are limited.

To address this problem, we propose the Reference-Regularized Complexity-Constrained Few-Shot Digital Twin Instantiation algorithm, termed RCF-DTI. An offline stage constructs a compact candidate bank of source-trained models with different architectures. For each target center, RCF-DTI removes candidates that exceed either complexity limit and adapts the remaining candidates using the same target data and a common update budget. It selects the best alternative only when its validation loss is sufficiently lower than that of a fixed reference candidate. The selected architecture and adapted parameters form the target-specific digital twin model. Multi-center workload prediction [17–19] is used only for evaluation.

The main contributions are as follows.

- The proposed formulation joins architecture selection and parameter adaptation in one target-specific digital twin instantiation problem with limited target data and hard operation-count and parameter-count limits. It includes a fixed reference candidate and requires sufficient validation improvement before selecting an alternative.
- RCF-DTI provides an algorithmic solution over a finite candidate bank. It filters candidates before adaptation because model complexity is fixed by architecture. It uses a common adaptation budget to make candidate losses comparable and calibrates the selection threshold before target instantiation.
- The analysis establishes that every output satisfies both limits and that every selected alternative reaches the required validation improvement. When validation error is bounded, it gives a sufficient condition for lower expected target loss. It also derives the target-stage computational complexity.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents RCF-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

2. Related Work

2.1. Digital Twin Modeling and Platform Deployment

Digital twins maintain virtual models of physical entities to support monitoring, prediction, simulation, and control [1–3]. In industrial and networked systems, data-driven models capture system states and support management decisions [4–6]. Digital twin networks extend this setting to network elements, computing nodes, and services, whose models may be hosted on cloud or edge platforms while remaining associated with individual physical entities [7, 8]. Platform-side studies further consider service placement and computation offloading under delay and resource constraints [27, 28].

These studies establish the basis for platform-side digital twin modeling and deployment, but they commonly assume a predefined or shared model architecture. They provide limited support for selecting different architectures for newly observed heterogeneous centers under target-specific complexity limits.

2.2. Few-Shot Learning and Meta-Learning

Few-shot learning adapts a model to a new task using limited target data, while meta-learning learns transferable knowledge, initializations, or update rules from related tasks [9, 10]. Context-aware and cross-domain methods improve transfer under changing tasks and data conditions [20, 21]. Prompt-based and memory-based methods provide task-specific information or stored experience [22, 23]. Related ideas have also been applied to digital twin construction, online and continual adaptation, and few-shot workload prediction [11, 24, 25].

Despite these advances, most methods adapt parameters within a fixed model architecture. They provide limited support for jointly selecting an architecture and adapting its parameters during target-specific digital twin model instantiation.

2.3. Deployment-Aware Architecture Selection

Neural architecture search selects model structures from a predefined architecture space. Platform-aware and hardware-aware methods consider both prediction accuracy and execution cost [12, 13]. Few-shot search shares information among candidate architectures to reduce repeated training [14], while zero-shot and training-free methods rank architectures without fully training every candidate [15, 16]. Multi-objective search further considers several accuracy and complexity measures during architecture selection [26].

These methods mainly improve search efficiency, architecture ranking, or accuracy–complexity trade-offs. They provide limited support for reliable target-specific model selection under hard complexity limits when target and validation data are limited.

Overall, existing work mainly treats platform-side digital twin management, few-shot adaptation, and deployment-aware architecture selection as separate problems. It provides limited support for joint architecture selection and parameter adaptation under hard operation-count and parameter-count limits, especially when validation data are limited. RCF-DTI addresses these limitations by filtering infeasible candidates, adapting feasible candidates with a common update budget, and requiring sufficient improvement over a fixed reference.

3. System Model and Problem Formulation

3.1. Platform Setting

We consider a shared edge–cloud digital twin platform that maintains model instances for heterogeneous computing centers. Let C_{src} denote the source centers that provide historical data for offline training. During each instantiation process, we consider one new or reconfigured target center c with limited target data.

Each physical center is associated with one model instance hosted by the shared platform. The centers may differ in monitored variables, workload patterns, and data distributions. They may therefore require different model architectures.

Let $\mathcal{A}$ denote the predefined architecture space. For an architecture $a \in \mathcal{A}$, let $f_{a,\theta}$ denote a data-driven digital twin model with parameter vector θ . The digital twin model instance for target center c is represented by

$$\text{DT}_c = (a_c, \theta_c), \quad (1)$$

where a_c is the selected model architecture and θ_c is its target-adapted parameter vector. Model instantiation must therefore determine both the architecture and the model parameters.

3.2. Candidate Models and Complexity Limits

The target observations are divided in chronological order into a support set $\mathcal{D}_c^{\text{sup}}$ and a validation set $\mathcal{D}_c^{\text{val}}$. The support set is used for target adaptation. The validation set is used for candidate selection.

A candidate model is denoted by $q = (a_q, \theta_q^0)$. Here, a_q is the candidate architecture and θ_q^0 is its source-trained initialization. We call a finite set of candidate models a candidate bank. Let $\mathcal{Q}$ denote the candidate bank constructed before target instantiation. Different candidates may use different architectures or different source-trained initializations.

The platform assigns two hard complexity limits to the model instance for target center c . These limits may vary with the available capacity at the hosting location and the service requirements. The first limit controls the computation required for one inference, and the second controls the model size. For architecture a_q , let $F_c(a_q)$ denote its estimated operation count for one inference under the input setting of center c . Let $P(a_q)$ denote its parameter count. Their assigned upper bounds are denoted by B_c^F and B_c^P , respectively.

The feasible candidate set is

$$\mathcal{Q}_c^{\text{fea}} = \{q \in \mathcal{Q} \mid F_c(a_q) \leq B_c^F, P(a_q) \leq B_c^P\}. \quad (2)$$

A candidate belongs to $\mathcal{Q}_c^{\text{fea}}$ only when it satisfies both limits. The two limits are hard constraints. They are not added to the prediction loss as penalty terms.

One candidate q^{ref} is fixed as the reference candidate before target instantiation. We assume that $q^{\text{ref}} \in \mathcal{Q}_c^{\text{fea}}$ for each target center to which the algorithm is applied. The reference candidate provides a fixed default choice when an alternative candidate does not provide enough validation improvement.

3.3. Problem Formulation

Let $\mathcal{L}(\mathcal{D}; a, \theta)$ denote the mean task loss of model $f_{a,\theta}$ on dataset $\mathcal{D}$. The task loss used in the experiments is given in Section 5.

Let T denote the fixed number of target parameter updates. For each feasible candidate q , let $\theta_{c,q}^{(T)}$ denote its parameter vector after T updates on $\mathcal{D}_c^{\text{sup}}$. The concrete update rule is given in Section 4. The post-adaptation validation loss is

$$J_c(q) = \mathcal{L}(\mathcal{D}_c^{\text{val}}; a_q, \theta_{c,q}^{(T)}). \quad (3)$$

Directly returning the candidate with the lowest value of $J_c(q)$ may be unreliable when the validation set is small. A small difference on the validation set may disappear on unseen target data. We therefore add a relative selection cost when a non-reference candidate is selected. We call this cost reference regularization.

Let $\tau \in [0, 1)$ denote the required relative improvement over the reference candidate. The regularized selection score is defined as

$$S_c(q; \tau) = \begin{cases} J_c(q), & q = q^{\text{ref}}, \\ J_c(q) + \tau J_c(q^{\text{ref}}), & q \neq q^{\text{ref}}. \end{cases} \quad (4)$$

The added term is used only for candidate selection. It does not change the parameter adaptation loss. A larger value of τ requires a larger validation improvement before the algorithm selects a non-reference candidate.

The target-specific instantiation problem is

$$q_c^* \in \arg \min_{q \in \mathcal{Q}_c^{\text{fea}}} S_c(q; \tau). \quad (5)$$

The formulation has two levels. The inner level adapts the parameter vector of each feasible candidate using the target support set. The outer level selects one candidate under the complexity limits using Eq. (4). We therefore treat target-specific model instantiation as a constrained two-level decision problem.

4. Proposed Method

4.1. Overview

RCF-DTI uses a candidate bank prepared offline together with a fixed reference candidate and a calibrated selection threshold. For each target center, the algorithm receives the target data and assigned complexity limits. It first removes candidates that violate either limit, then adapts the feasible candidates with a fixed update budget, and finally performs reference-regularized selection. The selected architecture and its adapted parameters form the target-specific digital twin instance. Figure 2 summarizes the overall RCF-DTI workflow and its relation to digital twin operation. The offline bank preparation and the target-side stages are detailed in the following subsections.

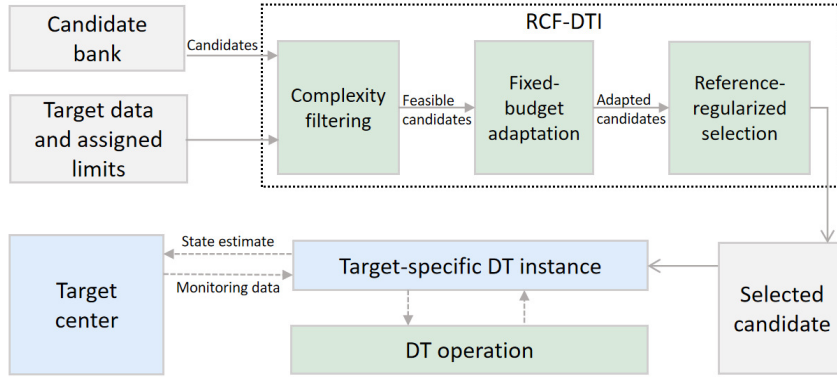


Figure 2: Overall workflow of RCF-DTI and its connection to target-center digital twin operation.

4.2. Offline Candidate Bank Construction

The candidate-bank input in Figure 2 is prepared offline in two steps. First, a compact architecture set is retained from the predefined space using separate development data. All architectures are compared using the same adaptation and evaluation procedure. The exact retention rule is reported in Section 5.1.

Let $\mathcal{A}_{\text{bank}} = \{a_m\}_{m=1}^M$ denote the retained architecture set, where M is the number of retained architectures. Second, let $\mathcal{D}_{\text{src}}$ denote the pooled source-center training data. For each retained architecture a_m , the offline stage trains one or more source initializations. The r -th initialization is trained by

$$\theta_{m,r}^0 \in \arg \min_{\theta} \mathcal{L}(\mathcal{D}_{\text{src}}; a_m, \theta), \quad m = 1, \dots, M, \quad r = 1, \dots, R_m, \quad (6)$$

where θ is the parameter variable during source training, r indexes the stored initializations for architecture a_m , and R_m is their total number. Different values of r distinguish different source-trained initializations obtained for the same architecture. In practice, the source optimizer gives an approximate solution to Eq. (6).

Each pair $q_{m,r} = (a_m, \theta_{m,r}^0)$ forms one candidate. The complete bank is $\mathcal{Q} = \{q_{m,r}\}$ for $m = 1, \dots, M$ and $r = 1, \dots, R_m$. Because each initialization is trained with its own architecture, it is called an architecture-matched source initialization.

The reference candidate q^{ref} is fixed from this bank before target instantiation and is not changed across target centers. Figure 3 details the offline construction of the candidate bank shown in Figure 2.

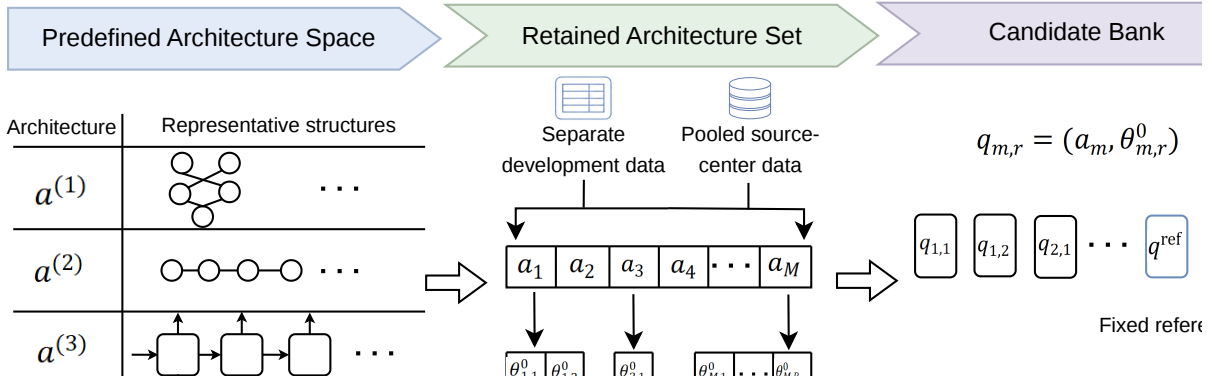


Figure 3: Offline candidate bank construction, including architecture retention, source-initialization training, and designation of a fixed reference candidate.

4.3. Candidate Filtering

For target center c , RCF-DTI computes the feasible set using Eq. (2). Filtering compares the estimated operation count $F_c(a_q)$ and parameter count $P(a_q)$ with their assigned upper bounds B_c^F and B_c^P , respectively. It does not use the target support or validation data. Because architecture complexity does not change during parameter adaptation, a candidate that violates either limit is removed before target updates. This avoids unnecessary adaptation. The feasible reference candidate ensures at least one valid output. Figure 4 illustrates representative complexity-filtering outcomes.

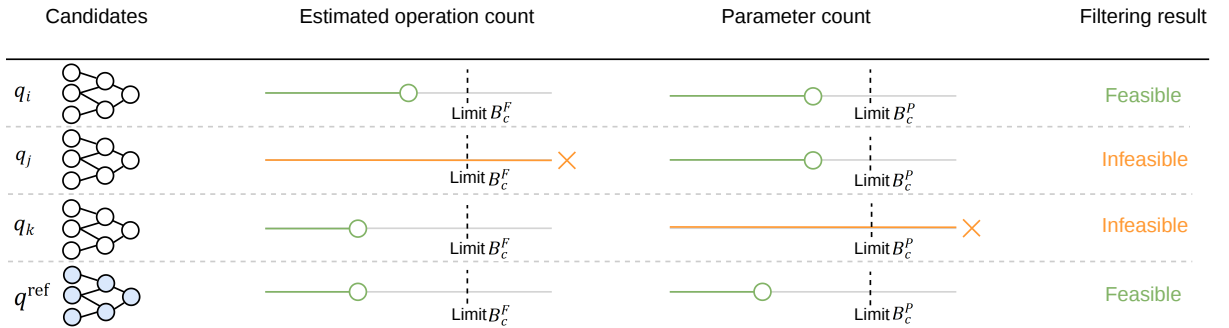


Figure 4: Representative candidate-filtering outcomes under the assigned operation-count and parameter-count limits.

4.4. Target Adaptation

Each feasible candidate starts from its architecture-matched source initialization. Let $\eta_t > 0$ denote the step size at update t . Its parameters are updated by

$$\begin{aligned} \theta_{c,q}^{(0)} &= \theta_q^0, \\ \theta_{c,q}^{(t+1)} &= \theta_{c,q}^{(t)} - \eta_t \nabla_{\theta} \mathcal{L} \left(D_c^{\text{sup}}; a_q, \theta_{c,q}^{(t)} \right), \\ t &= 0, \dots, T-1. \end{aligned} \quad (7)$$

Here, t is the update index, η_t is the step size, and ∇_{θ} denotes the gradient with respect to the model parameters. The architecture a_q remains fixed during adaptation. All feasible candidates use the same support set, task loss, and update budget T . These common conditions make their post-adaptation losses comparable.

After the final update, the algorithm computes $J_c(q)$ using Eq. (3). The validation set is used only for candidate selection. The optimizer and other implementation settings are reported in Section 5.1.

4.5. Candidate Selection

Let $\mathcal{Q}_c^{\text{alt}} = \mathcal{Q}_c^{\text{fea}} \setminus \{q^{\text{ref}}\}$ denote the set of feasible alternatives. If this set is empty, the algorithm returns the reference candidate.

Otherwise, let $\tilde{q}_c$ denote a feasible alternative with the lowest post-adaptation validation loss:

$$\tilde{q}_c \in \arg \min_{q \in Q_c^{\text{alt}}} J_c(q).$$

The final candidate is selected by

$$q_c^* = \begin{cases} \tilde{q}_c, & J_c(\tilde{q}_c) \leq (1 - \tau)J_c(q^{\text{ref}}), \\ q^{\text{ref}}, & \text{otherwise.} \end{cases} \quad (8)$$

Equation (8) returns a minimizer of the score-based problem in Eq. (5) because every alternative receives the same added selection cost.

Figure 5 illustrates the fixed-budget adaptation and reference-regularized selection stages.

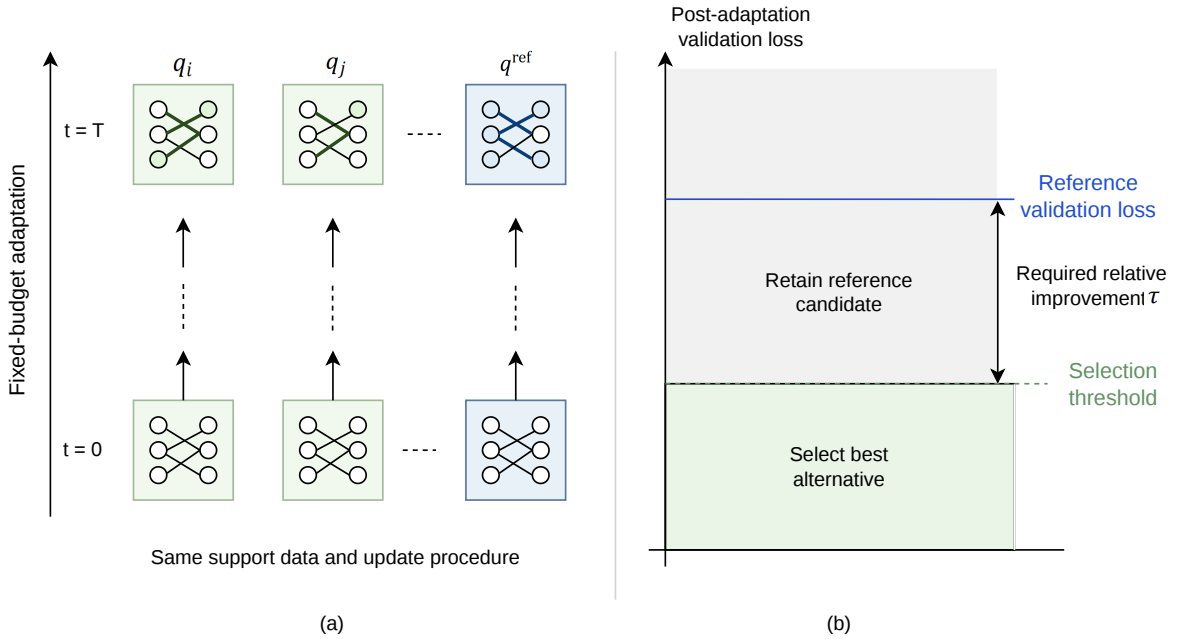


Figure 5: Fixed-budget target adaptation and reference-regularized selection. The best feasible alternative is selected when $J_c(\tilde{q}_c) \leq (1 - \tau)J_c(q^{\text{ref}})$; otherwise, the fixed reference candidate is retained.

Algorithm 1 summarizes the target-side execution of RCF-DTI for one target center.

4.6. Threshold Calibration

The threshold τ is fixed before target instantiation. It is selected from a finite set $\mathcal{T}$ using a separate set of development cases, denoted by C_{dev} . Each development case contains support, validation, and check data. The support and validation data are used during candidate adaptation and selection. The check data are not used for within-case adaptation or candidate selection. They are used only in offline development procedures, including architecture retention and threshold calibration.

For each case $d \in C_{\text{dev}}$, let $h_d(\tau) = 1$ when RCF-DTI selects a non-reference candidate whose check loss is higher than that of the reference candidate. Otherwise, let $h_d(\tau) = 0$. The empirical harmful-selection rate is

$$\hat{R}_{\text{harm}}(\tau) = \frac{1}{|C_{\text{dev}}|} \sum_{d \in C_{\text{dev}}} h_d(\tau). \quad (9)$$

Algorithm 1 RCF-DTI for Target-Specific Model Instantiation

Require: Candidate bank $\mathcal{Q}$, reference candidate q^{ref} , support set $\mathcal{D}_c^{\text{sup}}$, and validation set $\mathcal{D}_c^{\text{val}}$
Require: Limits B_c^F and B_c^P , update budget T , step sizes $\{\eta_t\}_{t=0}^{T-1}$, and calibrated threshold τ
Ensure: Target-specific digital twin model instance DT_c^*

- 1: Compute $\mathcal{Q}_c^{\text{fea}}$ using Eq. (2)
- 2: **for all** $q \in \mathcal{Q}_c^{\text{fea}}$ **do**
- 3: $\theta_{c,q}^{(0)} \leftarrow \theta_q^0$
- 4: **for** $t = 0$ to $T - 1$ **do**
- 5: $g_t \leftarrow \nabla_{\theta} \mathcal{L} \left(\mathcal{D}_c^{\text{sup}}; a_q, \theta_{c,q}^{(t)} \right)$
- 6: $\theta_{c,q}^{(t+1)} \leftarrow \theta_{c,q}^{(t)} - \eta_t g_t$
- 7: **end for**
- 8: Compute $J_c(q)$ using Eq. (3)
- 9: **end for**
- 10: $\mathcal{Q}_c^{\text{alt}} \leftarrow \mathcal{Q}_c^{\text{fea}} \setminus \{q^{\text{ref}}\}$
- 11: **if** $\mathcal{Q}_c^{\text{alt}} = \emptyset$ **then**
- 12: $q_c^* \leftarrow q^{\text{ref}}$
- 13: **else**
- 14: Choose $\tilde{q}_c \in \arg \min_{q \in \mathcal{Q}_c^{\text{alt}}} J_c(q)$
- 15: **if** $J_c(\tilde{q}_c) \leq (1 - \tau) J_c(q^{\text{ref}})$ **then**
- 16: $q_c^* \leftarrow \tilde{q}_c$
- 17: **else**
- 18: $q_c^* \leftarrow q^{\text{ref}}$
- 19: **end if**
- 20: **end if**
- 21: $\text{DT}_c^* \leftarrow (a_{q_c^*}, \theta_{c,q_c^*}^{(T)})$
- 22: **return** DT_c^*

Calibration also requires positive development gains. Let $\hat{G}_{\text{ref}}(\tau)$ denote the mean relative check-loss reduction over the fixed reference candidate. To distinguish gains from architecture selection from gains caused only by a different source initialization, we define a reference-architecture control. This control applies the same selection rule only to candidates that use the reference architecture. Let $\hat{G}_{\text{arch}}(\tau)$ denote the mean additional reduction over this control.

Let $\underline{G}_{\text{ref}}(\tau)$ and $\underline{G}_{\text{arch}}(\tau)$ denote the lower confidence bounds of the two gains. Let ρ denote the allowed harmful-selection rate, and let γ_{ref} and γ_{arch} denote the required minimum gains. The eligible threshold set and the calibrated threshold are

$$\begin{aligned}
 \mathcal{T}_{\text{elig}} = \{ \tau \in \mathcal{T} \mid & \hat{R}_{\text{harm}}(\tau) \leq \rho, \\
 & \hat{G}_{\text{ref}}(\tau) \geq \gamma_{\text{ref}}, \quad \underline{G}_{\text{ref}}(\tau) > 0, \\
 & \hat{G}_{\text{arch}}(\tau) \geq \gamma_{\text{arch}}, \quad \underline{G}_{\text{arch}}(\tau) > 0 \}, \\
 \tau^* = \min \mathcal{T}_{\text{elig}}.
 \end{aligned} \tag{10}$$

We assume that $\mathcal{T}_{\text{elig}}$ is nonempty. The smallest eligible threshold is used because it satisfies the development criteria without requiring a larger validation improvement than necessary. The tested values, gain requirements, and confidence procedure are reported in Section 5.1.

This calibration controls the observed behavior on the development cases. It does not guarantee improvement for every unseen target center.

4.7. Properties and Computational Complexity

Complexity feasibility. Algorithm 1 selects a candidate only from $\mathcal{Q}_c^{\text{fea}}$. Therefore, its output always satisfies

$$F_c(a_{q_c^*}) \leq B_c^F, \quad P(a_{q_c^*}) \leq B_c^P.$$

This property follows directly from the hard filtering step.

Minimum validation improvement. Suppose that the algorithm returns a non-reference candidate and that $J_c(q^{\text{ref}}) > 0$. Equation (8) gives

$$\frac{J_c(q^{\text{ref}}) - J_c(q_c^*)}{J_c(q^{\text{ref}})} \geq \tau.$$

Thus, every accepted alternative reaches at least the required relative validation improvement.

Selection under bounded validation error. Let $R_c(q)$ denote the expected loss of the adapted candidate q on future observations from target center c . Suppose that the validation loss approximates this expected loss with error at most ϵ_c for the selected alternative and the reference candidate:

$$|R_c(q) - J_c(q)| \leq \epsilon_c, \quad q \in \{q_c^*, q^{\text{ref}}\}.$$

If RCF-DTI selects a non-reference candidate, then Eq. (8) gives

$$R_c(q_c^*) - R_c(q^{\text{ref}}) \leq -\tau J_c(q^{\text{ref}}) + 2\epsilon_c. \quad (11)$$

Therefore, when $\tau J_c(q^{\text{ref}}) > 2\epsilon_c$, the selected alternative has a lower expected loss than the reference candidate on future target observations. This is a sufficient condition under the stated validation error bound. It does not provide an unconditional guarantee for every target center.

Target-stage computation. Let C_q^{grad} denote the cost of one gradient update for candidate q , and let C_q^{val} denote its validation cost. The target-stage computation is

$$O\left(|Q| + \sum_{q \in Q_c^{\text{fea}}} \left(T C_q^{\text{grad}} + C_q^{\text{val}}\right)\right). \quad (12)$$

Here, $O(\cdot)$ denotes asymptotic computational complexity. Given the complexity values available at filtering time, the first term is the filtering cost. The remaining terms are the adaptation and validation costs of the feasible candidates. Filtering before adaptation avoids the update cost of candidates that cannot satisfy the platform-assigned limits.

The algorithm returns DT_c^* and records $F_c(a_{q_c^*})$ and $P(a_{q_c^*})$ for the selected architecture.

5. Experimental Evaluation

This section evaluates average prediction error, performance on difficult cases, limit satisfaction, target-side cost, and generalization. It also examines the main components and configuration choices of RCF-DTI.

5.1. Experimental Setup

5.1.1. Benchmark and Evaluation Protocol

We use a controlled synthetic benchmark for heterogeneous multi-center workloads. The benchmark varies sampling intervals, missing-data patterns, noise levels, workload dynamics, target data sizes, and model complexity limits. Each center contains 3000 workload observations. The input window length is $L = 96$.

The main evaluation uses 20 source centers and 20 held-out target centers. For each target center, we consider two prediction horizons, $H \in \{1, 4\}$, and two support sizes, $K \in \{10, 20\}$. A target case is one combination of a target center, a prediction horizon, and a support size. The main evaluation therefore contains 80 target cases.

The target centers are grouped into types A, B, and C. These groups differ in sampling intervals, missing-data patterns, noise levels, and workload dynamics. The group label is used only for result analysis. It is not an input to RCF-DTI or any baseline.

We use four disjoint center groups: source, development, diagnostic, and held-out target. Source centers train the source initializations. Development centers are used for architecture retention and threshold calibration. The diagnostic group contains three disjoint pools for the ablation, optimizer-matched, and source-scale studies. Held-out target centers are used only for the main comparison.

Each target case is divided in chronological order into support, validation, check, and test sets. Support data are used for adaptation, and validation data are used for candidate selection. On development cases, validation and check data support architecture retention, and check data also calibrate the threshold. On diagnostic and held-out cases, check data are used only after candidate selection. Test data are used only for final evaluation. Neither check nor test data affect target adaptation or within-case candidate selection.

The reference architecture is a one-layer gated recurrent unit (GRU) with 32 hidden units and a dropout rate of 0.1. Its parameters are pretrained on the pooled source-center data. The resulting source-trained model is used as the reference candidate in all later experiments.

An alternative architecture is retained if, in at least two development cases, its error is lower than that of the reference architecture on the validation set or the check set. This rule fixes the retained architecture set before any diagnostic or held-out evaluation.

The selection threshold is fixed on the development cases. A harmful selection occurs when RCF-DTI selects a non-reference candidate whose check loss is higher than that of the reference candidate. We test thresholds of 5%, 7.5%, 10%, 12.5%, 15%, and 20%.

A threshold is eligible only when all development criteria are satisfied. First, its harmful-selection rate must not exceed 5%. Second, its mean check-loss reduction over the reference candidate must be at least 3%, and the lower bound of its 95% confidence interval must be positive. The additional reduction over the reference-architecture control must satisfy the same gain and confidence requirements. The smallest eligible threshold is 10%. The main candidate bank, reference candidate, and selection threshold are then fixed. They are not changed using the ablation, optimizer-matched, or held-out results. Table 1 lists the main experimental configuration.

Table 1: Experimental configuration.

Category	Item	Setting
Data	Source / held-out target centers	20 / 20
	Observations per center; input length	3000; $L = 96$
	Main paired cases	80
Target	Horizons; support sizes	$H \in \{1, 4\}; K \in \{10, 20\}$
	Validation / check / test windows	80 / 60 / 200
Model	Full architecture space; retained architectures	66 architectures; six retained architectures
	Candidate models	7 before filtering
	Feasible candidates	4–7 after filtering; mean 6.25
	Reference architecture	One-layer GRU; 32 hidden units; dropout 0.1
	Reference initialization	Pretrained on pooled source-center data
Source training	Optimizer / epochs	Adam / 50
	Learning rate / batch size	10^{-3} / 64
Target adaptation	Optimizer / loss / update budget	SGD / MSE / 50 steps
	Data used in each update	Complete target support set
	Learning rate / gradient clipping	0.01 / 1.0
Selection	Selection rule	Select an alternative candidate only when its validation MSE is at least 10% lower than that of the reference candidate
	Threshold calibration	Finite development grid; smallest value satisfying the harmful-selection, gain, and confidence criteria
Model complexity limits	Tight: estimated operation count; parameter count	1.5×10^6 / 3×10^4
	Medium: estimated operation count; parameter count	5×10^6 / 10^5
	Loose: estimated operation count; parameter count	2×10^7 / 5×10^5
Statistics	Confidence interval	Center-cluster bootstrap; 4000 repeats
Protocol	Test usage	Used only after architecture selection and parameter adaptation are complete

5.1.2. Task, Metrics, and Implementation

We use workload prediction only as the evaluation task. It tests whether an instantiated digital twin model captures the temporal behavior of the target center.

Let $\mathbf{X}_{c,t}$ denote an input window of length L . The model predicts the next H workload states as

$$\hat{\mathbf{y}}_{c,t+1:t+H} = f_{a,\theta}(\mathbf{X}_{c,t}), \quad (13)$$

where t is the current time index, $\hat{\mathbf{y}}_{c,t+1:t+H}$ is the predicted workload vector, and $\mathbf{y}_{c,t+1:t+H}$ is the corresponding observed vector.

We use mean squared error (MSE) as the task loss. For a dataset $\mathcal{D}$ of input–output pairs, it is defined as

$$\mathcal{L}(\mathcal{D}; a, \theta) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{X}, \mathbf{y}) \in \mathcal{D}} \frac{1}{H} \|\mathbf{f}_{a,\theta}(\mathbf{X}) - \mathbf{y}\|_2^2. \quad (14)$$

Here, $|\mathcal{D}|$ is the number of prediction windows, H is the prediction horizon, and $\|\cdot\|_2$ is the Euclidean norm. The loss averages the squared error over all prediction windows and prediction steps. It is the task loss introduced in Section 3.

We report mean absolute error (MAE), MSE, Worst-10% error, conditional value at risk at the 90th percentile (CVaR90), target-side instantiation time, and limit satisfaction. For each target case, MAE and MSE are averaged over all test windows and prediction steps. Worst-10% error is the mean window-level MSE over the 10% of test windows with the largest errors. CVaR90 is the mean case-level MSE over the 10% of target cases with the largest MSE values. Limit satisfaction is the percentage of selected models that meet both assigned model complexity limits.

The main results report averages over the held-out target cases. Since all methods are evaluated on the same target cases, relative gains are first computed for each paired case. The four (H, K) cases from the same target center are treated as one group. The 95% confidence intervals are obtained by resampling target centers 4000 times.

All methods are implemented in Python and PyTorch and run on the same NVIDIA RTX 3060 Laptop GPU. The main source initializations are trained with the Adam optimizer for 50 epochs. The learning rate is 10^{-3} , and the batch size is 64.

Each target update uses the complete support set. The updates are implemented with the SGD optimizer and MSE loss. All feasible RCF-DTI candidates use the same learning rate, gradient clipping rule, and fixed budget of 50 updates. Candidate-specific early stopping is not used.

GPU execution is synchronized before runtime measurement. The timer includes candidate model setup, loading of source initializations, target adaptation, validation evaluation, and final candidate selection. It excludes source training, data generation, and post-selection check and test evaluation.

The main candidate bank uses source-training seed 2904. Seeds 2905 and 2906 are used only in the additional training-seed study.

5.1.3. Baselines and Comparison Settings

We compare RCF-DTI with six baselines: pretraining and fine-tuning (PT+FT), MeDeT-based adaptation, random initialization, few-shot neural architecture search, zero-shot neural architecture search, and zero-shot neural architecture search with fine-tuning.

PT+FT adapts the source-trained reference candidate using the same SGD, MSE, and 50-step protocol as RCF-DTI. Random initialization uses the same reference architecture and adaptation protocol without source training. MeDeT-based adaptation uses source parameters learned with Model-Agnostic Meta-Learning (MAML) for the fixed reference architecture, following the few-shot adaptation setting of MeDeT [11]. Few-shot neural architecture search is an adapted comparison based on the candidate sharing and ranking setting in [14]. It retains and adapts the top 12 candidates. The two zero-shot baselines use architecture scores that do not require full target training [15, 16]. One selects a model without target adaptation. The other adapts its top 12 candidates before final selection.

The MeDeT and architecture-search baselines are adapted to the common workload prediction setting. They are not full reproductions of the original systems. All methods use the same target cases, chronological splits, complexity limits, and architecture definitions. No method uses the target test set for training, selection, or stopping.

PT+FT is the main controlled baseline because it adapts the same reference candidate as RCF-DTI. We also conduct an optimizer-matched comparison on a separate diagnostic pool. In this comparison, all adapted candidates use SGD,

MSE, and 50 updates, and each matched search baseline evaluates 12 candidates. The pool is not used to tune any method. Table 2 summarizes the comparison settings.

Table 2: Compared methods and target-side protocols.

Method	Structure	Source initialization	Target update	Adapted models	Complexity-limit check
PT+FT	Fixed reference architecture	Pooled source pretraining	SGD/MSE-50	1	Before output
MeDeT-based adaptation	Fixed reference architecture	Meta initialization	SGD/MSE-50	1	Before output
Random initialization	Fixed reference architecture	Random initialization	SGD/MSE-50	1	Before output
Few-shot NAS	Top-12 shortlist	Baseline-specific initialization bank	Adam/Huber-50	≤ 12	Before candidate evaluation
Zero-shot NAS	Top-ranked model	Baseline-specific initialization	None	0	Before candidate evaluation
Zero-shot NAS + fine-tuning	Top-12 shortlist	Baseline-specific initialization bank	Adam/Huber-50	≤ 12	Before candidate evaluation
RCF-DTI	Source-initialization bank (six architectures)	Architecture-matched source initializations	SGD/MSE-50	4–7	Before adaptation and output

PT+FT and RCF-DTI use the same source-pretrained reference model and the same target adaptation protocol.

5.2. Overall Performance

Table 3 reports the predictive performance on the held-out target cases. All compared methods use the same model complexity limits and the same feasibility check. Therefore, every selected model in this table satisfies both limits. The effect of removing complexity filtering is examined separately in Section 5.5.

Table 3: Overall predictive performance on the held-out target cases.

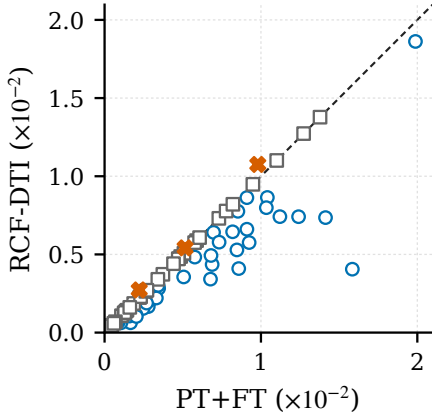
Method	MAE↓	MSE↓	Worst-10%↓	CVaR90↓
PT+FT	0.05382	0.005120	0.019078	0.013874
MeDeT-based adaptation	0.06409	0.007115	0.024170	0.022398
Random initialization	0.12562	0.026202	0.073298	0.086879
Few-shot NAS	0.06428	0.007105	0.026501	0.019089
Zero-shot NAS	0.13500	0.030387	0.075192	0.110634
Zero-shot NAS + fine-tuning	0.06654	0.007527	0.028539	0.018781
RCF-DTI	0.04866	0.004219	0.015755	0.011706

RCF-DTI gives the lowest MAE, MSE, Worst-10% error, and CVaR90 among all compared methods. PT+FT is the strongest baseline. Relative to PT+FT, RCF-DTI reduces MAE, MSE, and Worst-10% error by 8.55%, 14.60%, and 13.76%, respectively.

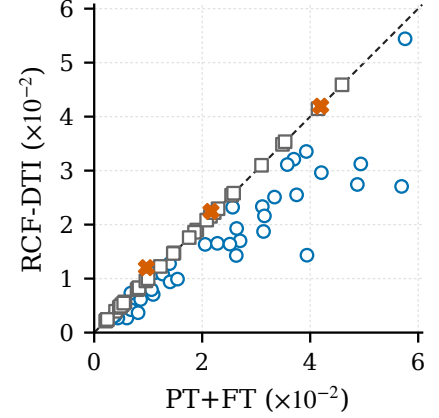
These reductions are first computed for each paired target case and then averaged. They are not computed from the aggregate values in Table 3. The corresponding 95% confidence intervals are [5.42%, 11.85%], [9.41%, 20.14%], and [8.44%, 19.25%]. All three intervals remain above zero.

We next examine the case-level results against PT+FT. Since PT+FT adapts the reference candidate, it also provides the test result of the reference model under the common target adaptation settings. For post-selection analysis, an alternative selection is called beneficial when its test MSE is lower than that of PT+FT. It is called harmful when its test MSE is higher. These labels are assigned after candidate selection using the test set. They do not affect the algorithm. Figure 6 shows the paired comparison. Points below the diagonal favor RCF-DTI.

○ Beneficial alternative (44) □ Reference retained (33) ✖ Harmful alternative (3)



(a) Test MSE



(b) Test Worst-10% error

Figure 6: Paired held-out results for RCF-DTI and PT+FT: (a) test MSE and (b) test Worst-10% error. Points below the diagonal favor RCF-DTI.

RCF-DTI selects an alternative in 47 cases. Among them, 44 are beneficial and 3 are harmful. Thus, 93.62% of the alternative selections improve test MSE, while harmful selections account for 3.75% of all held-out cases. The reference is retained in the other 33 cases. Reference regularization therefore reduces, but does not remove, harmful selections.

Table 4 reports the optimizer-matched comparison on the separate diagnostic target pool.

Table 4: Optimizer-matched comparison on a separate diagnostic target pool.

Method	MSE↓	Worst-10%↓	CVaR90↓	Adapted candidates
RCF-DTI	0.003809	0.015094	0.008696	6.10
PT+FT (reference model)	0.004497	0.017144	0.011878	1.00
Few-shot NAS (matched)	0.006715	0.025246	0.019653	12.00
Zero-shot NAS (matched)	0.007402	0.028099	0.020402	12.00
Common 12-candidate shortlist, lowest validation loss	0.006707	0.025240	0.019653	12.00
Common 12-candidate shortlist, reference-regularized selection	0.006819	0.025562	0.020547	12.00

RCF-DTI uses its fixed compact candidate bank and reference-regularized selection. No method is tuned on this pool.

RCF-DTI still gives the lowest MSE, Worst-10% error, and CVaR90. It adapts 6.10 candidates on average, while each matched search baseline adapts 12 candidates. RCF-DTI's advantage therefore remains after matching the optimizer, task loss, and number of target updates.

This comparison does not isolate one component of RCF-DTI. The compared methods still differ in their source initializations and candidate-selection procedures.

5.3. Performance under Target Heterogeneity

We first examine whether the gain of RCF-DTI is limited to one target setting. Figure 7 reports the paired MSE reduction relative to PT+FT under different prediction horizons, support sizes, and target-center types. A positive value means that RCF-DTI gives lower MSE than PT+FT. The confidence intervals are obtained by resampling target centers.

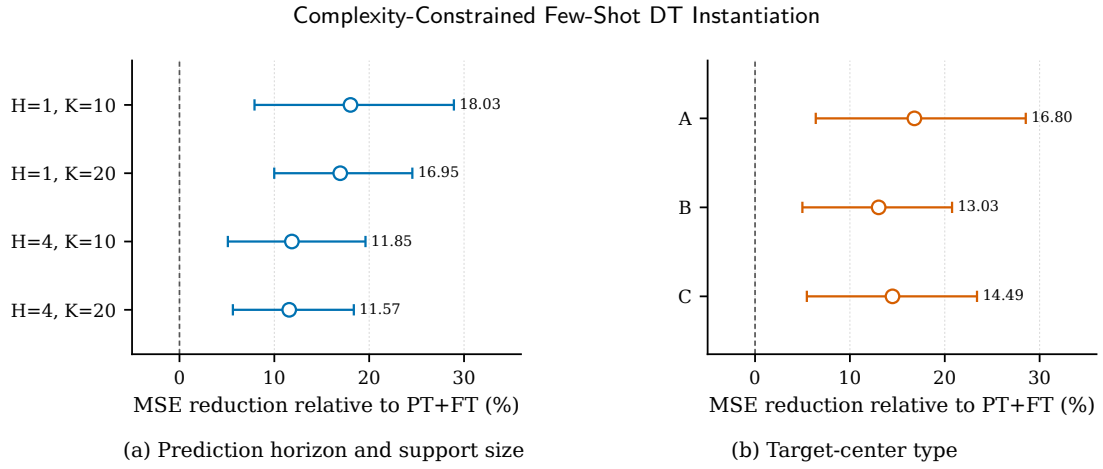


Figure 7: Paired MSE reduction relative to PT+FT across (a) prediction horizons and support sizes and (b) target-center types. Error bars show 95% confidence intervals obtained by resampling target centers.

RCF-DTI gives positive mean reductions in all four (H, K) settings and all three target-center types. The reductions range from 11.57% to 18.03% across the (H, K) settings and from 13.03% to 16.80% across the center types. All seven confidence intervals remain above zero. Thus, the gain is not limited to one prediction horizon, support size, or center type.

5.4. Effect of Complexity Limits

We next examine how the assigned complexity limits affect target-side instantiation. The benchmark includes tight, medium, and loose limit settings. The tight setting uses the smallest upper bounds on estimated operation count and parameter count. The medium setting uses intermediate bounds, and the loose setting uses the largest bounds.

Figure 8 compares the number of candidates before and after complexity filtering. It also reports the selection rates of the final model configurations under the three limit settings. Here, a model configuration refers to an architecture and its source initialization before target adaptation.

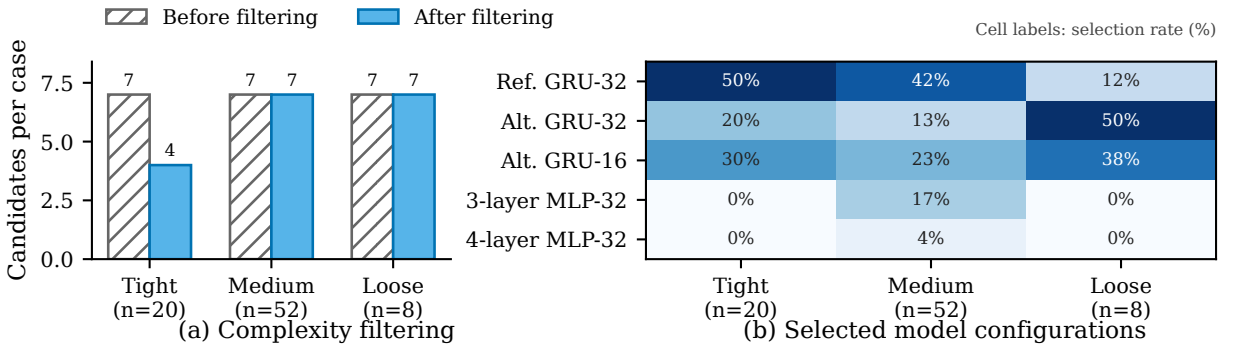


Figure 8: Effect of tight, medium, and loose complexity limits on (a) the number of feasible candidates and (b) final model selection.

Under the tight setting, filtering reduces the mean candidate count from seven to four. The medium and loose settings keep all seven candidates. Five model configurations are selected across the held-out cases. The tight setting selects only GRU configurations, while the medium setting also selects two multilayer perceptron configurations. Under the loose setting, the alternative GRU configurations are selected most often.

These results show that the assigned limits affect both adaptation cost and the final model choice. Because filtering is applied first, every selected model satisfies both limits.

5.5. Ablation Study

Table 5 reports the ablation results on an independent diagnostic target pool. This pool does not affect the main candidate bank, reference candidate, selection threshold, or other main settings. Gain is the paired MSE reduction relative to the reference candidate on the same pool.

Table 5: Ablation results on the independent diagnostic target pool. Gain is the paired MSE reduction relative to the reference candidate.

Variant	MSE↓	Worst-10%↓	Gain (%)↑	Harmful selection (%)↓	Limit satisfaction (%)↑
RCF-DTI	0.004063	0.015764	15.79	15.00	100.00
Earlier pooled-source training					
for non-reference models	0.004876	0.019044	1.75	2.50	100.00
Reference candidate only	0.005051	0.019512	0.00	0.00	100.00
Two reference-architecture initializations	0.004917	0.019136	1.29	1.25	100.00
Without reference regularization [†]	0.004055	0.015690	16.16	17.50	100.00
Without complexity filtering [†]	0.004037	0.015635	16.13	13.75	96.25

[†] Diagnostic variants only. The harmful-selection rate is computed over all diagnostic cases. Without complexity filtering, a selected model may exceed an assigned limit.

We first examine the source-training procedure and the role of architecture selection. The pooled-source variant keeps the candidate architectures and the two reference-architecture initializations unchanged, but replaces the other initializations with those produced by the earlier pooled-source procedure. The reference-only variant uses only the fixed reference candidate, while the two-initialization variant selects between the two source initializations of the reference architecture. All three variants give higher MSE than RCF-DTI.

These results indicate that both source-initialization training and selection among different architectures contribute to target adaptation. The pooled-source comparison changes the complete training procedure for the non-reference initializations and therefore does not isolate a single training factor.

Removing reference regularization slightly reduces mean MSE, but increases the harmful-selection rate from 15.00% to 17.50%. Both rates are higher than the 5% rate used during development calibration. Reference regularization therefore trades a small change in mean MSE for fewer harmful selections, although it does not remove them completely.

Removing complexity filtering slightly reduces mean MSE, but lowers limit satisfaction from 100% to 96.25%. Complexity filtering therefore enforces the assigned hard limits rather than directly reducing prediction error.

5.6. Configuration and Cost Analysis

Figure 9 examines three configuration choices: the number of retained architectures, the target update budget, and the selection threshold. Only development or diagnostic cases are used in these analyses.

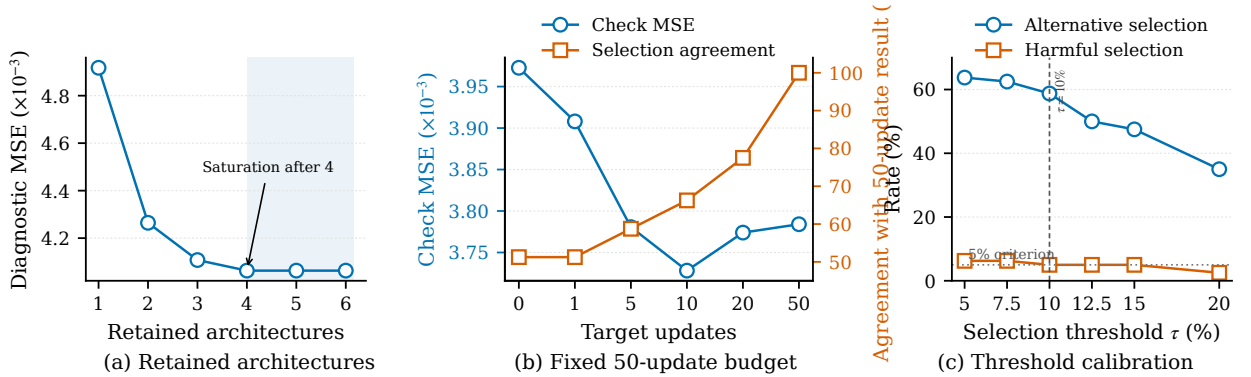


Figure 9: Configuration analysis using development or diagnostic cases: (a) retained architecture count, (b) adaptation budget, and (c) selection threshold. Held-out target cases are not used to choose these settings.

Figure 9(a) shows that diagnostic MSE decreases from 0.004917 with one retained architecture to 0.004063 with four retained architectures, and then remains unchanged. We retain all six architectures because the retained architecture set was fixed before the diagnostic results were observed. This result supports a compact retained architecture set, but does not identify a globally optimal size.

Figure 9(b) shows that check MSE decreases mainly during the first ten updates. Here, selection agreement is the percentage of cases in which a given update budget selects the same candidate as the 50-update setting. At 20 updates, selection agreement reaches 77.5%. The mean case-level Spearman rank correlation between the candidate rankings at 20 and 50 updates is 0.913. We use 50 updates as a fixed common budget, not as a claimed optimal value. Figure 9(c)

shows how the alternative-selection and harmful-selection rates change with the threshold. The 10% threshold is the smallest tested value that satisfies all development criteria defined in Section 5.1. It is then fixed for held-out evaluation.

Figure 10 compares RCF-DTI with PT+FT and two architecture-search baselines using normalized target-side measures. The normalized scores are used only for visualization; all comparisons are based on the raw values.

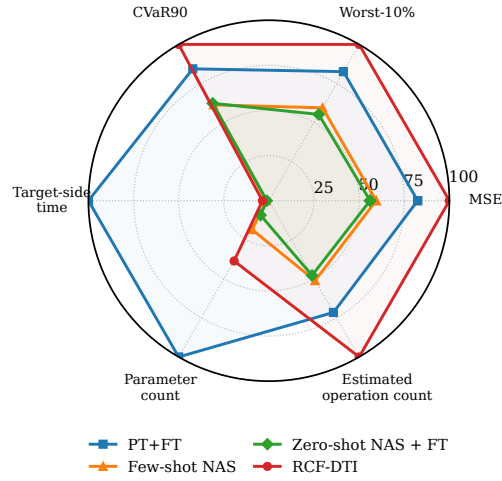


Figure 10: Normalized target-side trade-offs of RCF-DTI and representative baselines. For each lower-is-better measure x , the displayed score is $100x_{\min}/x$, where $x_{\min}$ is the lowest raw value among the displayed methods. Larger scores are better.

RCF-DTI gives the lowest MSE, Worst-10% error, and CVaR90 among the displayed methods, but it is not the fastest. It adapts 6.25 candidates on average and takes 5.676 ± 0.059 seconds per case. Single-model methods are faster because they adapt one candidate. Few-shot NAS and zero-shot NAS with fine-tuning are slower because they adapt 12 candidates. Their target-side times are $2.37\times$ and $3.10\times$ that of RCF-DTI, respectively.

Figure 11 examines whether the overall gain is concentrated in one selected model configuration. Test results are used only for post-selection analysis.

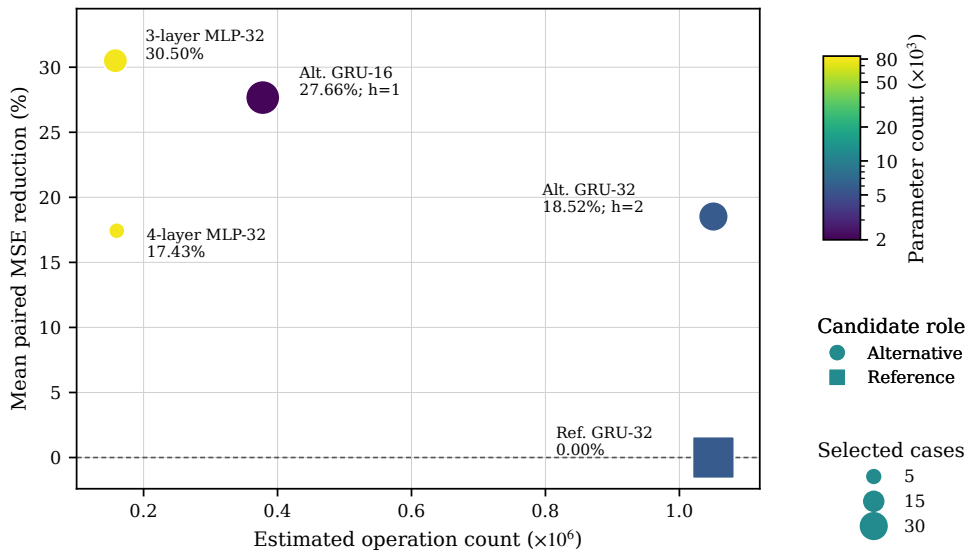


Figure 11: Complexity and test performance of the selected model configurations. Marker area denotes the number of selected cases, and h denotes the number of harmful selections.

The five selected configurations show different operation-count and parameter-count trade-offs. The two multilayer perceptron configurations use fewer estimated operations but more parameters than the GRU configurations. The

reference configuration is retained in 33 cases, while four alternative configurations cover the remaining 47 cases. Their mean paired MSE reductions range from 17.43% to 30.50%.

All four alternatives give positive mean reductions, so the overall gain is not produced by one fixed architecture. The three harmful selections occur on the two alternative GRU configurations. Reference-regularized selection therefore reduces, but does not remove, harmful selections.

5.7. Generalization Study

We examine RCF-DTI under changes in source-center scale, source-training seed, and workload data. These studies use the same target-side adaptation and selection procedure as the main evaluation. In each study, RCF-DTI is compared with a reference candidate trained under the same source condition and source-training budget. We call this candidate the matched reference.

Source-center scale. We rebuild the candidate bank using 10, 20, 30, 40, and 50 source centers. Each source-trained model receives 2000 source-training updates at every scale. At each scale, RCF-DTI and the matched reference use the same source-center set and source-training budget. The evaluation uses the separate source-scale diagnostic target pool.

Figure 12(a) reports the paired MSE reduction relative to the matched reference at each source scale. The 20-source-center result belongs to this diagnostic study. It is therefore not directly comparable with the 14.60% main result, which is obtained on the held-out target centers.

The mean reduction is positive at all five source scales, but it does not increase monotonically. Under a fixed source-training budget, adding more source centers distributes the same number of updates across more source workloads. More source centers therefore do not always produce a larger target-side gain.

Source-training seeds and external data. We rebuild the candidate bank using source-training seeds 2904, 2905, and 2906. The three banks use the same source centers, architecture set, source-training budget, and data-generation seed. They are evaluated on one fixed diagnostic target pool with identical target adaptation seeds and the same target-side protocol. The only changed factor is the source-training seed. This study therefore examines the sensitivity of the candidate bank to source-training randomness.

We also apply the fixed RCF-DTI configuration to machine-usage data from Alibaba Cluster Trace v2018 [29]. The workload observations are real. The 20 target machines are assigned deterministically to the tight, medium, and loose complexity settings in groups of 6, 8, and 6, respectively, using a fixed hash-based ordering of machine identifiers. These settings use the same operation-count and parameter-count bounds as the synthetic benchmark. The candidate bank, adaptation procedure, filtering rule, and selection threshold are not returned.

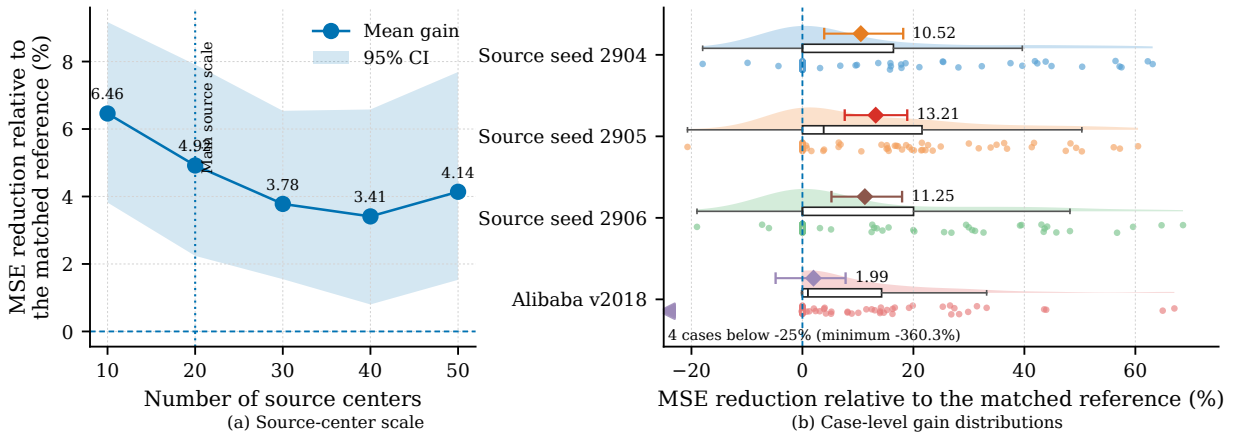


Figure 12: Generalization under changed source conditions. (a) Paired MSE reduction relative to the reference trained at each source scale; shading shows 95% confidence intervals. (b) Case-level reductions for three source-training seeds and Alibaba v2018. Four Alibaba cases below -25% are clipped only for display; all original values are retained in the analysis.

Across the three source-training seeds, the mean paired MSE reduction ranges from 10.52% to 13.21%, and all three confidence intervals remain above zero. Some cases still have zero or negative gains.

On Alibaba v2018, the mean reduction is 1.99%, but its confidence interval includes zero. The distribution is also wider, with four cases below -25% . Thus, the fixed RCF-DTI configuration can be applied without retuning, but it does not show a consistent improvement on this dataset.

6. Conclusion

This paper studied complexity-constrained few-shot digital twin model instantiation for heterogeneous multi-center computing networks. We formulated target-specific instantiation as joint architecture selection and parameter adaptation under limits on the estimated operation count per inference and the parameter count. We then proposed RCF-DTI as an algorithmic solution. RCF-DTI filters infeasible candidates, adapts the feasible candidates with a common update budget, and retains the fixed reference unless an alternative provides sufficient validation improvement. Experiments show that RCF-DTI achieves lower prediction error than all compared methods while every selected model satisfies both limits. The mean gains remain positive across heterogeneous target settings and source-training seeds, whereas the confidence interval on Alibaba v2018 includes zero. Future work will evaluate actual latency, memory use, and energy consumption on deployed edge-cloud platforms.

CRedit Authorship Contribution Statement

The final author contribution statement will be added after the author list is confirmed.

Funding

Funding information will be added after confirmation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The source code, frozen experimental configurations, processed results, plot-ready figure data, and reproduction scripts supporting this study are publicly available in the versioned GitHub Release v1.1.9: <https://github.com/w924871681/paper-code-DT-Twin/releases/tag/v1.1.9>. The corresponding development repository is available at <https://github.com/w924871681/paper-code-DT-Twin>.

The release contains the materials required to reproduce Tables 1–5 and the data-driven Figures 6–12 using the documented public entry point. Figures 1–5 are provided as checksum-verified final manuscript assets. SHA-256 checksums are supplied for the release assets to support integrity verification. Synthetic multi-center data can be regenerated using the released simulator, frozen configurations, and fixed random seeds. The anonymized case-level values used in Figure 12(b) are included as plot-ready data.

The original Alibaba Cluster Trace v2018 is third-party data and is not redistributed in this repository. It is available from the official Alibaba Cluster Data repository: <https://github.com/alibaba/clusterdata>. The released documentation specifies the required source file and checksum, expected file layout, preprocessing procedure, source-bank construction, and evaluation commands. Publicly released Alibaba-derived records contain only anonymized identifiers and processed evaluation values.

References

- [1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2230–2254, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3198273.
- [2] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad, H. Venkataraman, R. Trestian, and H. X. Nguyen, "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, Fourth Quarter 2022, doi: 10.1109/COMST.2022.3208773.

- [3] H. Xu, J. Wu, Q. Pan, X. Guan, and M. Guizani, "A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2569–2598, Fourth Quarter 2023, doi: 10.1109/COMST.2023.3297395.
- [4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019, doi: 10.1109/TII.2018.2873186.
- [5] B. Qin, H. Pan, Y. Dai, X. Si, X. Huang, C. Yuen, and Y. Zhang, "Machine and deep learning for digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: 10.1109/JIOT.2024.3416733.
- [6] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging Internet of Things industry," *Computer Networks*, vol. 244, Art. no. 110350, 2024, doi: 10.1016/j.comnet.2024.110350.
- [7] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet of Things Journal*, vol. 8, no. 18, pp. 13789–13804, Sept. 2021, doi: 10.1109/JIOT.2021.3079510.
- [8] J. Li, S. Guo, W. Liang, J. Wang, Q. Chen, Y. Zeng, B. Ye, and X. Jia, "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 6, pp. 7335–7350, 2024, doi: 10.1109/TMC.2023.3332668.
- [9] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögvaldsson, and K. C. Santosh, "Advances and challenges in meta-learning: A technical review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 7, pp. 4763–4779, 2024, doi: 10.1109/TPAMI.2024.3357847.
- [10] H. Gharoun, F. Momenifar, F. Chen, and A. H. Gandomi, "Meta-learning approaches for few-shot learning: A survey of recent advances," *ACM Computing Surveys*, vol. 56, no. 12, Art. no. 294, pp. 1–41, 2024, doi: 10.1145/3659943.
- [11] H. Sartaj, S. Ali, and J. M. Gjølby, "MeDeT: Medical device digital twins creation with few-shot meta-learning," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 6, Art. no. 158, pp. 1–36, 2025, doi: 10.1145/3708534.
- [12] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 2820–2828, doi: 10.1109/CVPR.2019.00293.
- [13] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, and K. Keutzer, "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 10734–10742, doi: 10.1109/CVPR.2019.01099.
- [14] Y. Zhao, L. Wang, Y. Tian, R. Fonseca, and T. Guo, "Few-shot neural architecture search," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139 of *Proc. Mach. Learn. Res.*, pp. 12707–12718, 2021.
- [15] G. Li, D. Hoang, K. Bhardwaj, M. Lin, Z. Wang, and R. Marculescu, "Zero-shot neural architecture search: Challenges, solutions, and opportunities," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 7618–7635, Dec. 2024, doi: 10.1109/TPAMI.2024.3395423.
- [16] M.-T. Wu, H.-I. Lin, and C.-W. Tsai, "A training-free neural architecture search algorithm based on search economics," *IEEE Transactions on Evolutionary Computation*, vol. 28, no. 2, pp. 445–459, Apr. 2024, doi: 10.1109/TEVC.2023.3264533.
- [17] Y.-M. Kim, S. Song, B.-M. Koo, J. Son, Y. Lee, and J.-G. Baek, "Enhancing long-term cloud workload forecasting framework: Anomaly handling and ensemble learning in multivariate time series," *IEEE Transactions on Cloud Computing*, vol. 12, no. 2, pp. 789–799, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3400859.
- [18] F. Zhao, W. Lin, S. Lin, S. Tang, and K. Li, "MSCNet: Multi-scale network with convolutions for long-term cloud workload prediction," *IEEE Transactions on Services Computing*, vol. 18, no. 2, pp. 969–982, Mar.–Apr. 2025, doi: 10.1109/TSC.2025.3536313.
- [19] B. Feng and Z. Ding, "Application-oriented cloud workload prediction: A survey and new perspectives," *Tsinghua Science and Technology*, vol. 30, no. 1, pp. 34–54, Feb. 2025, doi: 10.26599/TST.2024.9010024.
- [20] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97 of *Proc. Mach. Learn. Res.*, pp. 7693–7702, 2019.
- [21] Y. Zhao, T. Zhang, J. Li, and Y. Tian, "Dual adaptive representation alignment for cross-domain few-shot learning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 10, pp. 11720–11732, 2023, doi: 10.1109/TPAMI.2023.3272697.
- [22] J. Wu, X. Liu, X. Yin, T. Zhang, and Y. Zhang, "Task-adaptive prompted transformer for cross-domain few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6012–6020, 2024, doi: 10.1609/aaai.v38i6.28416.
- [23] W. Wang, L. Duan, Y. Wang, J. Fan, and Z. Zhang, "MMT: Cross domain few-shot learning via meta-memory transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 15018–15035, Dec. 2023, doi: 10.1109/TPAMI.2023.3306352.
- [24] J. Son, S. Lee, and G. Kim, "When meta-learning meets online and continual learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 1, pp. 413–432, 2025, doi: 10.1109/TPAMI.2024.3463709.
- [25] Z. Zuo, Y. Huang, Z. Li, Y. Jiang, and C. Liu, "Mixed contrastive transfer learning for few-shot workload prediction in the cloud," *Computing*, vol. 107, no. 1, Art. no. 5, 2025, doi: 10.1007/s00607-024-01366-y.
- [26] Y. Zhao, L. Wang, and T. Guo, "Multi-objective neural architecture search by learning search space partitions," *Journal of Machine Learning Research*, vol. 25, no. 177, pp. 1–41, 2024.
- [27] M. E. Khansari and S. Sharifian, "A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum," *Journal of Network and Computer Applications*, vol. 233, Art. no. 104042, 2025, doi: 10.1016/j.jnca.2024.104042.
- [28] D. Hortelano, I. de Miguel, R. J. Durán Barroso, J. C. Aguado, N. Merayo, L. Ruiz, A. Asensio, X. Masip-Bruin, P. Fernández, R. M. Lorenzo, and E. J. Abril, "A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems," *Journal of Network and Computer Applications*, vol. 216, Art. no. 103669, 2023, doi: 10.1016/j.jnca.2023.103669.
- [29] [dataset] Alibaba Group, "Alibaba Cluster Trace Program: Cluster Trace v2018," GitHub repository, 2018. [Online]. Available: <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018>. Accessed: Jul. 3, 2026.